C++ COMPLETE REFERENCE LIBRARY • MODULE 05

05 FUNCTIONS — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 parameter passing value address reference
3. **Theory Chapter:** 02 default arguments inline and overloading
4. **Theory Chapter:** 03 recursion and stack frames
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

05 — Functions

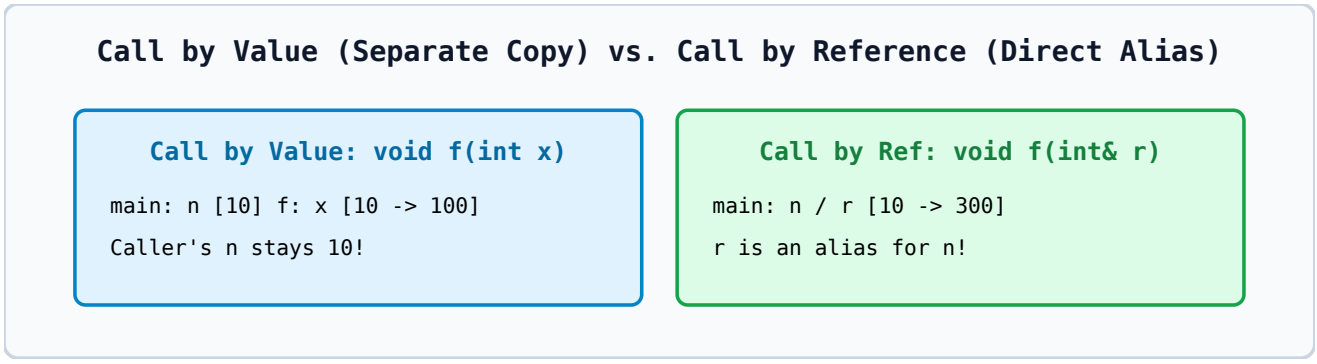
This module covers function declarations (prototypes) vs. definitions, Call by Value vs. Call by Address vs. Call by Reference, default arguments, `inline` functions, Function Overloading, and Recursion.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_parameter_passing_value_address_reference.md` (./01_THEORY/01_parameter_passing_value_address_reference.md)
  - `02_default_arguments_inline_and_overloading.md` (./01_THEORY/02_default_arguments_inline_and_overloading.md)
  - `03_recursion_and_stack_frames.md` (./01_THEORY/03_recursion_and_stack_frames.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_call_by_value_pointer_reference.cpp` (./02_PROGRAMS/01_call_by_value_pointer_reference.cpp) (Category A)
  - `02_function_overloading_and_default_args.cpp` (./02_PROGRAMS/02_function_overloading_and_default_args.cpp) (Category A)

  - `03_ambiguous_overload_compile_error.cpp` (`./02_PROGRAMS/03_ambiguous_overload_compile_error.cpp`) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_call_by_value_vs_reference.svg` (`./05_DIAGRAMS/diagram_01_call_by_value_vs_reference.svg`)

01 — Call by Value, Call by Address (Pointer), and Call by Reference



Call by Value vs Reference

Mechanism	Syntax	Copies Argument?	Can Modify Caller's Variable?
Call by Value	`void f(int x)`	**Yes** (independent local copy)	**No**
Call by Address	`void f(int* p)`	Copies pointer address (8 bytes)	**Yes** (via `*p = ...`)
Call by Reference	`void f(int& r)`	**No** (`r` aliases caller's variable)	**Yes** (via `r = ...`)
Call by `const` Reference	`void f(const string& s)`	**No**	**No** (Read-only, zero-copy)

02 — Default Arguments, `inline` Functions, and Function Overloading

1. Trailing Default Arguments Rule (`[dcl.fct.default]`)

Once a parameter is given a default argument in a function declaration, **all subsequent parameters to its right** must also have default arguments:

- `int sum(int a, int b = 10, int c = 20);` -> **Legal**
- `int bad(int a = 10, int b);` -> **Compilation Error!**

2. Function Overloading Rules

Two functions in the same scope can share the same name if and only if their **parameter-type-lists** differ (in number, types, or `const` qualification of reference/pointer targets).

- **Crucial Rule**: Functions **cannot** be overloaded based **solely on return type** (`int f(int)` and `double f(int)` in the same scope is a **Compilation Error**).

03 — Recursion and Call Stack Frames

Every recursive function call pushes a new activation record (stack frame) onto the call stack containing its own local variables and return address. Every recursive function **must** have a **base case** that terminates recursion; otherwise, infinite recursion exhausts stack memory (Stack Overflow).

Visual Concept & Memory Diagrams (05 FUNCTIONS)

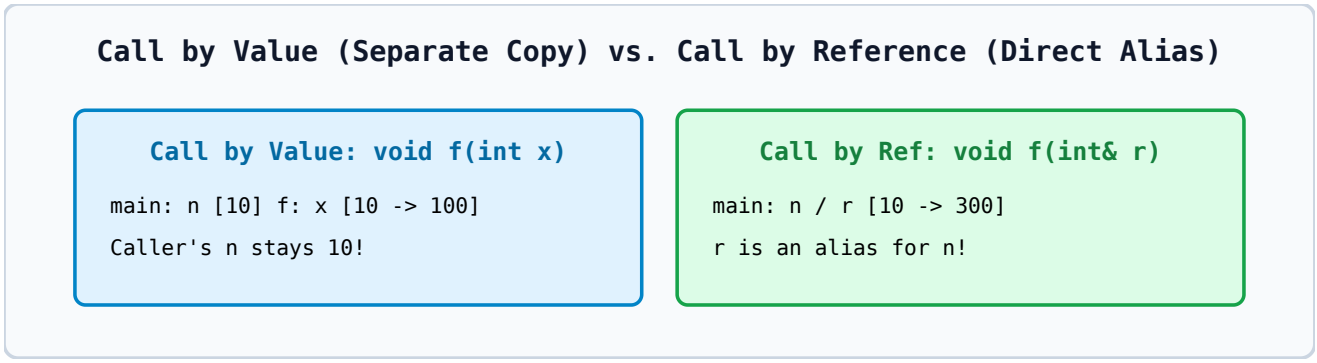


Diagram: diagram 01 call by value vs reference

Annotated C++ Programs (05 FUNCTIONS)

Program: `01_call_by_value_pointer_reference.cpp`

C++ SOURCE FILE • 01_call_by_value_pointer_reference.cpp

```cpp
// Program Title: Comparing Call by Value, Call by Address, and Call by Reference
// Topic: Functions — Parameter Passing
// Purpose: Show how each parameter-passing mode affects the caller's variable.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_parameter_passing_value_address_reference.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

void by_value(int x) { x = 100; }
void by_pointer(int* p) { *p = 200; }
void by_reference(int& r) { r = 300; }

int main() {
    int n = 10;
    by_value(n);
    cout << "After by_value: " << n << endl;

    by_pointer(&n);
    cout << "After by_pointer: " << n << endl;

    by_reference(n);
    cout << "After by_reference: " << n << endl;

    return 0;
}

/*
Expected Deterministic Output:
After by_value: 10
After by_pointer: 200
After by_reference: 300
*/
```

Program: `02_function_overloading_and_default_args.cpp`

C++ SOURCE FILE • 02_function_overloading_and_default_args.cpp

```cpp
// Program Title: Function Overloading and Default Arguments
// Topic: Functions — Overloading & Default Parameters
// Purpose: Demonstrate overloaded area() functions and trailing default arguments.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_default_arguments_inline_and_overloading.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int area(int side) {
    return side * side;
}

int area(int length, int breadth) {
    return length * breadth;
}

int compute_interest(int principal, int rate = 5) {
    return (principal * rate) / 100;
}

int main() {
    cout << "Square area: " << area(6) << endl;
    cout << "Rectangle area: " << area(6, 4) << endl;
    cout << "Default interest: " << compute_interest(1000) << endl;
    return 0;
}

/*
Expected Deterministic Output:
Square area: 36
Rectangle area: 24
Default interest: 50
*/
```

Program: `03_ambiguous_overload_compile_error.cpp`

C++ SOURCE FILE • 03_ambiguous_overload_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Ambiguous Function Overload With Default Arguments
// Topic: Functions — Overload Resolution Ambiguity
// Purpose: Show why overloading 'func(int)' and 'func(int, int = 0)' causes a compile-time
ambiguity on 'func(10)'.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_default_arguments_inline_and_overloading.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

void show(int a) {
    cout << a << endl;
}

void show(int a, int b = 0) {
    cout << a << " " << b << endl;
}

int main() {
    show(10); // COMPILE ERROR: call of overloaded 'show(int)' is ambiguous
    return 0;
}

/*
Why Compilation Fails:
Both 'show(int)' and 'show(int, int = 0)' can be called with a single int argument '10',
and neither is a better match than the other under ISO C++ [over.match].
*/
```

Conceptual Questions — Module 05: Functions

Q1. Can two functions in C++ be overloaded if they differ only in their return type? Why or why not? **Q2.** Explain the trailing default arguments rule in C++.

Debugging Questions — Module 05

Q4. Find the compile error in `int calc(int a = 5, int b, int c = 10);`.

Predict the Output Questions — Module 05

Q3. Predict the output of a recursive factorial function `fact(5)`.

Programming Questions — Module 05

Q5. Write a C++23 program using recursive greatest common divisor (`gcd(a, b)`).

Detailed Solutions — Conceptual Questions (Module 05)

- **Q1 Solution**: **No.** Overload resolution selects a function based on the arguments at the call site (`f(10)` may ignore the return value completely), so return type is not part of the function signature used for overloading.
- **Q2 Solution**: Once a parameter has a default argument, every parameter to its right must also have a default argument so positional arguments at the call site map unambiguously.

Detailed Solutions — Predict the Output & Debugging (Module 05)

- **Q3 Solution**: `fact(5) = 5 * 4 * 3 * 2 * 1 = 120`.
- **Q4 Solution**: Parameter `b` has no default argument even though `a` to its left has a default argument. Move `b` to the first position (`int calc(int b, int a = 5, int c = 10);`).

Detailed Solutions — Programming Questions (Module 05)

CPP

```cpp
#include <iostream>
using namespace std;
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
int main() {
    cout << "GCD(48, 18) = " << gcd(48, 18) << endl;
    return 0;
}
// Expected Output: GCD(48, 18) = 6
```